

👁 Watch 0

🍴 Fork 0

★

from vibe coding to agentic engineering - practice makes claude perfect

MIT License

[linkedin.com/in/shanraishan](#)

★ 1 star 🍴 0 forks 👁 0 watching 👤 1 branch 🏷 0 tags ↻ Activity

Public repository · Forked from [shanraishan/claude-code-best-practice](#)

m

1 Branch

0 Tags

Go to file

T

Go to file

Add file

Code

This branch is **919 commits behind** shanraishan/claude-code-best-practice:main .

Contribute

Sync fork

shanraishan and claude

chore(readme): refresh Last Updated badge timestamp to 2026-05-11 10:...

4s

4d

months ago

!	add video-2.png thumbnail	2 months ago
.claude	record presentation-claude-code Learnings f...	2 months ago
.codex	updated	3 months ago
.github	revert FUNDING.yml to custom checkout lin...	3 months ago
agent-teams	fix: add missing description frontmatter to ti...	3 months ago
best-practice	chore(claude-subagents): bump version bad...	2 months ago
changelog	chore(agent-collections): append 2026-05-1...	2 months ago
development-workflows	adeed claude-oss banner	4 months ago
implementation	updated weather workflow	3 months ago
orchestration-workflow	updated weather workflow	3 months ago
presentation	insert slide 29 Uncle Bob counterpoint + ren...	2 months ago
reports	Create claude-spinner-verbs-and-tips.md	3 months ago
tips	tips/claude-thariq: fix off-by-one cascade in ...	3 months ago
tutorial	add Day 1 onboarding tutorial — prompting, ...	3 months ago
videos	karparthy added	3 months ago
.gitignore	fix: add hooks log directory to .gitignore to p...	3 months ago
.mcp.json	fix: pin MCP server package versions to prev...	3 months ago
CLAUDE.md	rename presentation agent	3 months ago
LICENSE	updated docs	4 months ago
README.md	chore(readme): refresh Last Updated badge ...	2 months ago



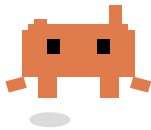
🔗 claude-code-best-practice

from vibe coding to agentic engineering - practice makes claude perfect

updated with Claude Code v2.1.138 (May 11, 2026 10:36 AM PKT) ★ 64k

Best Practice Implemented Orchestration Workflow Claude Boris Community Click on these badges below to see the actual sources

A = Agents · C = Commands · S = Skills



1 GITHUB TRENDING
#1 Repository Of The Day



Boris Cherny ✓
@bcherny



I'm Boris and I created Claude Code. Lots of people have asked how I use Claude Code, so I wanted to show off my setup a bit.

My setup might be surprisingly vanilla! Claude Code works great out of the box, so I personally don't customize it much. There is no one correct way to use Claude Code: we intentionally build it in a way that you can use it, customize it, and hack it however you like. Each person on the Claude Code team uses it *very* differently.

So, here goes.

12:58 AM · Jan 3, 2026 · 8.1M Views

Boris Cherny on X ([tweet 1](#) · [tweet 2](#) · [tweet 3](#))



Tip

Visit the [How to Use](#) section to take full advantage of this repo.

🔗 🧠 CONCEPTS

Feature	Location	Description
A Subagents	.claude/agents/<name>.md	Best Practice Implemented
C Commands	.claude/commands/<name>.md	Best Practice Implemented
S Skills	.claude/skills/<name>/SKILL.md	Best Practice Implemented Official Skills · Skills for Mono-repos
Workflows	.claude/commands/weather-orchestrator.md	Orchestration Workflow
Hooks	.claude/hooks/	Best Practice Implemented Guide
MCP Servers	.claude/settings.json , .mcp.json	Best Practice Implemented
Plugins	distributable packages	Marketplaces · Create Marketplaces
Settings	.claude/settings.json	Best Practice Implemented Permissions · Model Config · Output Styles · Sandboxing · Keybindings · Auto Mode Config

Feature	Location	Description
Status Line	<code>.claude/settings.json</code>	Best Practice Implemented
Memory	<code>CLAUDE.md</code> , <code>.claude/rules/</code> , <code>~/.claude/rules/</code> , <code>~/.claude/projects/<project>/memory/</code>	Best Practice Implemented Auto Memory · Auto Memory Deep-dive · Rules
Checkpointing	automatic (git-based)	
CLI Startup Flags	<code>claude [flags]</code>	Best Practice Interactive Mode · Env Vars
AI Terms		Best Practice
Best Practices		Prompt Engineering · Extend Claude Code

 Hot

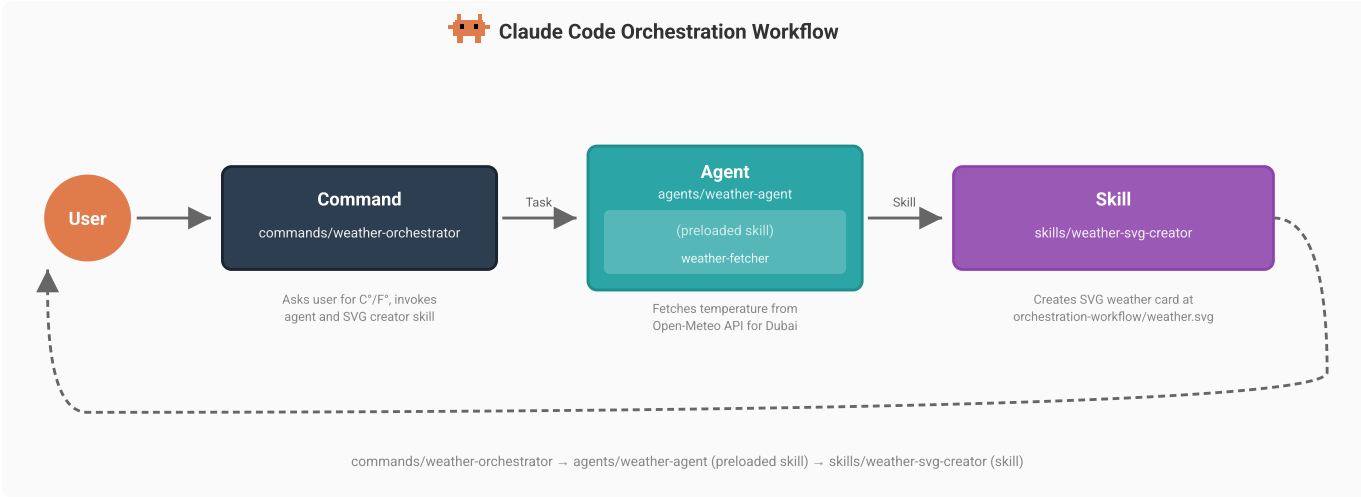
Feature	Location	Description
Ultrareview beta	<code>/ultrareview</code> , <code>claude ultrareview [target]</code>	Tasks tracking
Devcontainers	<code>.devcontainer/</code>	
Channels beta	<code>--channels</code> , plugin-based	Reference
Ultraplan beta	<code>/ultraplan</code>	
No Flicker Mode beta	<code>/tui fullscreen</code> , <code>CLAUDE_CODE_NO_FLICKER=1</code>	Best Practice
Auto Mode beta	<code>--permission-mode auto</code> , <code>Shift+Tab</code>	Best Practice Blog
Power-ups	<code>/powerup</code>	Best Practice
Fast Mode beta	<code>/fast</code> , <code>"fastMode": true</code>	
Computer Use beta	<code>computer-use</code> MCP server	Desktop
Agent SDK	<code>npm</code> / <code>pip</code> package	Quickstart · Examples
Ralph Wiggum Loop	plugin	Best Practice Implemented
Chrome beta	<code>--chrome</code> , extension	Best Practice
Claude Code Web beta	<code>claude.ai/code</code>	Routines
Slack	@Claude in Slack	
Code Review beta	GitHub App (managed)	Best Practice Blog
GitHub Actions	<code>.github/workflows/</code>	GitLab CI/CD
Remote Control	<code>/remote-control</code> , <code>/rc</code>	Best Practice Headless Mode
Agent Teams beta	built-in (env var)	Best Practice Implemented
Scheduled Tasks	<code>/loop</code> , <code>/schedule</code> , cron tools	Best Practice Implemented Desktop scheduled tasks · Announcement
Routines beta	<code>claude.ai/code/routines</code> , <code>/schedule</code>	Desktop Tasks
Tasks	<code>/tasks</code> , <code>~/.claude/tasks/</code>	Best Practice Ultrareview tracking
Voice Dictation beta	<code>/voice</code>	Best Practice
Simplify & Batch	<code>/simplify</code> , <code>/batch</code>	Best Practice

Feature	Location	Description
Git Worktrees	built-in, EnterWorktree / ExitWorktree , isolation: "worktree"	Best Practice



🔗 [Orchestration Workflow](#)

See [orchestration-workflow](#) for implementation details of **C** Command → **A** Agent → **S** Skill pattern.



```
shayanraees@DLB-DEV-SHAYANRAEES claudes-code-best-practice % claude
```

How to Use

```
claude
/weather-orchestrator
```



🔗 ⚙️ **DEVELOPMENT WORKFLOWS**

All major workflows converge on the same architectural pattern: **Research** → **Plan** → **Execute** → **Review** → **Ship**

Name	★	Workflow	A	C	S
Superpowers	184k	brainstorming → using-git-worktrees → writing-plans → subagent-driven-development → test-driven-development → requesting-code-review → verification-before-completion → finishing-a-development-branch	0	0	14
Everything Claude Code	176k	prp-prd → prp-plan → prp-implement → build-fix → code-review → quality-gate → prp-commit → prp-pr	48	71	182

Name	★	Workflow	A	C	S
Spec Kit	94k	<code>/speckit.constitution</code> → <code>/speckit.specify</code> → <code>/speckit.plan</code> → <code>/speckit.tasks</code> → <code>/speckit.implement</code> → <code>/speckit.checklist</code> → <code>/speckit.taskstoissues</code>	0	9	0
gstack	92k	<code>/office-hours</code> → <code>/plan-ceo-review</code> → <code>/plan-eng-review</code> → <code>/plan-design-review</code> → <code>/plan-devex-review</code> → <code>/autoplan</code> → <code>/review</code> → <code>/qa</code> → <code>/ship</code> → <code>/land-and-deploy</code> → <code>/canary</code>	0	0	56
Matt Pocock Skills	67k	<code>/grill-with-docs</code> → <code>/to-prd</code> → <code>/tdd</code> → <code>/diagnose</code> → <code>/zoom-out</code> → <code>/improve-codebase-architecture</code> → <code>/to-issues</code> → <code>/triage</code>	0	0	13
Get Shit Done	61k	<code>/gsd-map-codebase</code> → <code>/gsd-new-project</code> → <code>/gsd-discuss-phase</code> → <code>/gsd-plan-phase</code> → <code>/gsd-execute-phase</code> → <code>/gsd-verify-work</code> → <code>/gsd-ship</code> → <code>/gsd-complete-milestone</code> → <code>/gsd-new-milestone</code>	33	66	0
BMAD-METHOD	47k	<code>bmad-agent-analyst</code> → <code>bmad-prfaq</code> → <code>bmad-agent-pm</code> → <code>bmad-create-prd</code> → <code>bmad-agent-ux-designer</code> → <code>bmad-agent-architect</code> → <code>bmad-create-architecture</code> → <code>bmad-create-epics-and-stories</code> → <code>bmad-agent-dev</code> → <code>bmad-dev-story</code> → <code>bmad-code-review</code> → <code>bmad-retrospective</code>	6	0	40
OpenSpec	47k	<code>/opsx:explore</code> → <code>/opsx:propose</code> → <code>/opsx:apply</code> → <code>/opsx:verify</code> → <code>/opsx:sync</code> → <code>/opsx:archive</code>	0	11	0
oh-my-claudecode	33k	<code>/omc-setup</code> → <code>/deepinit</code> → <code>/plan</code> → <code>/autopilot</code> → <code>/ultrawork</code> → <code>/verify</code> → <code>/ultraqa</code> → <code>/release</code>	19	27	38
agent-skills	27k	<code>/spec</code> → <code>/plan</code> → <code>/build</code> → <code>/test</code> → <code>/review</code> → <code>/ship</code>	3	7	21
Compound Engineering	16k	<code>/ce-strategy</code> → <code>/ce-ideate</code> → <code>/ce-brainstorm</code> → <code>/ce-plan</code> → <code>/ce-work</code> → <code>/ce-debug</code> → <code>/ce-code-review</code> → <code>/ce-doc-review</code> → <code>/ce-compound</code> → <code>/ce-product-pulse</code>	49	4	38
HumanLayer	11k	<code>/research_codebase</code> → <code>/create_plan</code> → <code>/validate_plan</code> → <code>/iterate_plan</code> → <code>/implement_plan</code> → <code>/local_review</code> → <code>/describe_pr</code> → <code>/commit</code> → <code>/create_handoff</code>	6	27	0

Note: yellow tags are sub-loops – steps that repeat inside a parent step (e.g. per task, per story, or until a verify condition passes).

Others

- [Cross-Model \(Claude Code + Codex\) Workflow](#) Implemented
- [RPI](#) Implemented
- [Ralph Wiggum Loop](#) Implemented
- [Andrei Karpathy \(Founding Member, OpenAI\) Workflow](#)
- [Peter Steinberger \(Creator of OpenClaw\) Workflow](#)
- Boris Cherny (Creator of Claude Code) Workflow – [13 Tips](#) · [10 Tips](#) · [12 Tips](#) · [2 Tips](#) · [15 Tips](#) · [6 Tips](#) ✶ Boris
- Thariq (Anthropic) Workflow – [Skills](#) · [Session Management](#) ✶ Thariq



SKILL COLLECTIONS

Repos primarily known as curated libraries of SKILL.md files (distinct from full workflow methodologies above). Sorted by stars descending.

Name	★	S
anthropics/skills	131k	17
mattpocock/skills	68k	19
agent-skills	37k	22
wshobson/agents	35k	153

Name	★	S
awesome-agent-skills	21k	938 (curated list)
scientific-agent-skills	20k	137



🔗 🤖 AGENT COLLECTIONS

Repos primarily known as curated libraries of subagent definitions (`.claude/agents/*.md`). Sorted by stars descending.

Name	★	A
msitarzewski/agency-agents	96k	185
VoltAgent/awesome-claude-code-subagents	20k	145



🔗 💡 TIPS AND TRICKS (83)

🚫 🤖 = do not babysit

[Prompting](#) · [Planning](#) · [Context](#) · [Session](#) · [CLAUDE.md + .claude/rules](#) · [Agents](#) · [Commands](#) · [Skills](#) · [Hooks](#) · [Workflows](#) · [Advanced](#) · [Git / PR](#) · [Debugging](#) · [Utilities](#) · [Daily](#)

Community

■ Prompting (3)

Tip	Source
challenge Claude — "grill me on these changes and don't make a PR until I pass your test." or "prove to me this works" and have Claude diff between main and your branch 🚫 🤖	🌟 Boris
after a mediocre fix — "knowing everything you know now, scrap this and implement the elegant solution" 🚫 🤖	🌟 Boris
Claude fixes most bugs by itself — paste the bug, say "fix", don't micromanage how 🚫 🤖	🌟 Boris

■ Planning/Specs (7)

Tip	Source
always start with plan mode	🌟 Boris
start with a minimal spec or prompt and ask Claude to interview you using AskUserQuestion tool, then make a new session to execute the spec	🌟 Thariq
always make a phase-wise gated plan, with each phase having multiple tests (unit, automation, integration)	Dex Video
break PRDs into vertical slices (tracer bullets) that cross all layers (DB + service + UI) — AI defaults to horizontal phasing (DB phase, then API phase, then frontend phase) which delays end-to-end feedback until the last phase. From the Pragmatic Programmer 🚫 🤖	Matt Video
spin up a second Claude to review your plan as a staff engineer, or use cross-model for review	🌟 Boris
write detailed specs and reduce ambiguity before handing work off — the more specific you are, the better the output	🌟 Boris
prototype > PRD — build 20-30 versions instead of writing specs, the cost of building is low so take many shots	🌟 Boris Video

■ Context (5)

Tip	Source
context rot kicks in around ~300-400k tokens on the 1M context model — don't let sessions drift past that for intelligence-sensitive work	✨ Thariq
dumb zone kicks in around ~40% context — "you hit this point where you have degrading results". Newcomers: "shoot to keep it under 40%, and if you get up to 60%, think about wrapping it up". Experienced: "aggressively keep it below 30%" — push to 60% only on simple tasks. Manual /compact or /clear to reset when switching tasks	Dex Video
rewind > correct — double-Esc or /rewind back to before the failed attempt and re-prompt with what you learned, instead of leaving failed attempts + corrections polluting context 🚫🤔	✨ Thariq
/compact with a hint (/compact focus on the auth refactor, drop the test debugging) beats letting autocompact fire — the model is at its least intelligent point when auto-compacting due to context rot	✨ Thariq
use subagents for context management — ask yourself "will I need this tool output again, or just the conclusion?" — 20 file reads + 12 greps + 3 dead ends stay in the child's context, only the final report returns 🚫🤔	✨ Thariq

■ Session Management (6)

Tip	Source
every turn is a branching point — after Claude ends a turn, pick between Continue, /rewind , /clear , /compact , or Subagent based on how much existing context you need to carry forward	✨ Thariq
new task = new session — related tasks (e.g. writing docs for what you just built) can reuse context for efficiency, but genuinely new tasks deserve a fresh session	✨ Thariq
use "summarize from here" before rewinding to have Claude write a handoff message — like a note to the previous iteration of Claude from its future self	✨ Thariq
/compact vs /clear — compact is lossy but momentum-friendly (mid-task, fuzzy details ok); /clear + brief is more work but you control exactly what carries forward (high-stakes next step)	✨ Thariq
use recaps for long-running sessions — short summaries of what Claude did and what's next, useful when returning after minutes or hours. Disable with recaps in /config	✨ Boris
/rename important sessions (e.g. [TODO - refactor task]) and /resume them later — label each instance when running multiple Claudes simultaneously	✨ Cat

■ CLAUDE.md + .claude/rules (8)

Tip	Source
CLAUDE.md should target under 200 lines per file. 60 lines in humanlayer (still not 100% guaranteed)	✨ Boris Dex
.claude/rules/*.md auto-load into every session like CLAUDE.md — add paths: YAML frontmatter to lazy-load them only when Claude touches files matching the glob	✨ Claude
wrap domain-specific CLAUDE.md rules in <important if="..."> tags to stop Claude from ignoring them as files grow longer	Dex
use multiple CLAUDE.md for monorepos — ancestor + descendant loading	✨ Boris
use .claude/rules/ to split large instructions	✨ Claude
any developer should be able to launch Claude, say "run the tests" and it works on the first try — if it doesn't, your CLAUDE.md is missing essential setup/build/test commands	Dex
keep codebases clean and finish migrations — partially migrated frameworks confuse models that might pick the wrong pattern	✨ Boris Video
use settings.json for harness-enforced behavior (attribution, permissions, model) — don't put "NEVER add Co-Authored-By" in CLAUDE.md when attribution.commit: "" is deterministic	davila7

🔗 Agents (4)

Tip	Source
have feature specific sub-agents (extra context) with skills (progressive disclosure) instead of general qa, backend engineer	 Boris
say "use subagents" to throw more compute at a problem — offload tasks to keep your main context clean and focused 🚫🤖	 Boris
agent teams with tmux and git worktrees for parallel development	 Boris
use test time compute — separate context windows make results better; one agent can cause bugs and another (same model) can find them	 Boris

📌 Commands (3)

Tip	Source
use commands for your workflows instead of sub-agents	 Boris
use slash commands for every "inner loop" workflow you do many times a day — saves repeated prompting, commands live in .claude/commands/ and are checked into git	 Boris
if you do something more than once a day, turn it into a skill or command — build /techdebt, context-dump, or analytics commands	 Boris

📌 Skills (9)

Tip	Source
use context: fork to run a skill in an isolated subagent — main context only sees the final result, not intermediate tool calls. The agent field lets you set the subagent type	 Lydia
use skills in subfolders for monorepos	 Claude
skills are folders, not files — use references/, scripts/, examples/ subdirectories for progressive disclosure	 Thariq
build a Gotchas section in every skill — highest-signal content, add Claude's failure points over time	 Thariq
skill description field is a trigger, not a summary — write it for the model ("when should I fire?")	 Thariq
don't state the obvious in skills — focus on what pushes Claude out of its default behavior 🚫🤖	 Thariq
don't railroad Claude in skills — give goals and constraints, not prescriptive step-by-step instructions 🚫🤖	 Thariq
include scripts and libraries in skills so Claude composes rather than reconstructs boilerplate	 Thariq
embed !command in SKILL.md to inject dynamic shell output into the prompt — Claude runs it on invocation and the model only sees the result	 Lydia

■ Hooks (5)

Tip	Source
use on-demand hooks in skills — /careful blocks destructive commands, /freeze blocks edits outside a directory	 Thariq
measure skill usage with a PreToolUse hook to find popular or undertriggering skills	 Thariq
use a PostToolUse hook to auto-format code — Claude generates well-formatted code, the hook handles the last 10% to avoid CI failures	 Boris
route permission requests to Opus via a hook — let it scan for attacks and auto-approve safe ones 🚫🤖	 Boris
use a Stop hook to nudge Claude to keep going or verify its work at the end of a turn	 Boris

■ Workflows (5)

Tip	Source
use /model to select model and reasoning, /context to see context usage, /usage to check plan limits, /extra-usage to configure overflow billing, /config to configure settings — use Opus for plan mode and Sonnet for code to get the best of both	 Cat

Tip	Source
always use thinking mode true (to see reasoning) and Output Style Explanatory (to see detailed output with ★ Insight boxes) in /config for better understanding of Claude's decisions	 Boris
use ultrathink keyword in prompts for high effort reasoning	 Claude
<code>/focus</code> mode hides all intermediate work and shows only the final result — trust the model to run the right commands and just look at the outcome (toggle with <code>/focus</code>)	 Boris
tune effort level with Opus 4.7's adaptive thinking — low for speed and fewer tokens, max for most intelligence (slider: low · medium · high · xhigh · max)	 Boris

■ Workflows Advanced (9)

Tip	Source
use ASCII diagrams a lot to understand your architecture	 Boris
use /loop for local recurring monitoring (up to 7 days) · use /schedule for cloud-based recurring tasks that run even when your machine is off	 Boris
use Ralph Wiggum plugin for long-running autonomous tasks	 Boris
/permissions with wildcard syntax (Bash(npm run *), Edit(/docs/**)) instead of dangerously-skip-permissions	 Boris
/sandbox to reduce permission prompts with file and network isolation — 84% reduction internally	 Boris  Cat
invest in product verification skills (signup-flow-driver, checkout-verifier) — worth spending a week to perfect	 Thariq
use auto mode instead of dangerously-skip-permissions — a model-based classifier decides if each command is safe and auto-approves, pauses and asks if risky. Shift+Tab to cycle Ask → Plan → Auto modes 🚫🤖	 Boris
use <code>/less-permission-prompts</code> skill to scan session history for safe bash/MCP commands that repeatedly prompt, then get a recommended allowlist to paste into settings	 Boris
build a <code>/go</code> skill that (1) tests end-to-end via bash/browser/computer use (2) runs <code>/simplify</code> (3) puts up a PR — so when you come back, you know the code works 🚫🤖	 Boris

■ Git / PR (5)

Tip	Source
keep PRs small and focused — p50 of 118 lines (141 PRs, 45K lines changed in a day), one feature per PR, easier to review and revert	 Boris
always squash merge PRs — clean linear history, one commit per feature, easy git revert and git bisect	 Boris
commit often — try to commit at least once per hour, as soon as task is completed, commit	 Shayan
tag @claude on a coworker's PR to auto-generate lint rules for recurring review feedback — automate yourself out of code review 🚫🤖	 Boris  Video
use /code-review for multi-agent PR analysis — catches bugs, security vulnerabilities, and regressions before merge	 Boris

■ Debugging (6)

Tip	Source
make it a habit to take screenshots and share with Claude whenever you are stuck with any issue	 Shayan
use mcp (Claude in Chrome , Playwright , Chrome DevTools) to let claude see chrome console logs on its own	 Claude
always ask claude to run the terminal (you want to see logs of) as a background task for better debugging	 Shayan
/doctor to diagnose installation, authentication, and configuration issues	 Shayan

Tip	Source
use a cross-model for QA — e.g. Codex for plan and implementation review	 Shayan
agentic search (glob + grep) beats RAG — Claude Code tried and discarded vector databases because code drifts out of sync and permissions are complex	 Boris  Video

■ Utilities (5)

Tip	Source
iTerm/Ghostty/tmux terminals instead of IDE (VS Code/Cursor)	 Boris
/voice or Wispr Flow for voice prompting (10x productivity)	 Boris
claude-code-hooks for claude feedback	 Shayan
status line for context awareness and fast compacting	 Boris  Shayan
explore settings.json features like Plans Directory , Spinner Verbs for a personalized experience	 Boris

■ Daily (2)

Tip	Source
update Claude Code daily	 Shayan
start your day by reading the changelog	 Shayan



Article / Tweet	Source
6 Tips for Getting More Out of Opus 4.7 (Boris) 16/Apr/26	Tweet
Session Management & 1M Context (Thariq) 16/Apr/26	Tweet
15 Hidden & Under-Utilized Features in Claude Code (Boris) 30/Mar/26	Tweet
Squash Merging & PR Size Distribution (Boris) 25/Mar/26	Tweet
Lessons from Building Claude Code: How We Use Skills (Thariq) 17/Mar/26	Article
Code Review & Test Time Compute (Boris) 10/Mar/26	Tweet
/loop — schedule recurring tasks for up to 3 days (Boris) 07 Mar 2026	Tweet
AskUserQuestion + ASCII Markdowns (Thariq) 28 Feb 2026	Tweet
Seeing like an Agent - lessons from building Claude Code (Thariq) 28 Feb 2026	Article
Git Worktrees - 5 ways how boris is using 21 Feb 2026	Tweet
Lessons from Building Claude Code: Prompt Caching Is Everything (Thariq) 20 Feb 2026	Article
12 ways how people are customizing their claudes (Boris) 12/Feb/26	Tweet
10 tips for using Claude Code from the team (Boris) 01/Feb/26	Tweet
How I use Claude Code — 13 tips from my surprisingly vanilla setup (Boris) 03/Jan/26	Tweet
Ask Claude to interview you using AskUserQuestion tool (Thariq) 28/Dec/25	Tweet
Always use plan mode, give Claude a way to verify, use /code-review (Boris) 27/Dec/25	Tweet

🔗 Tips from Claude code CLI binary

[Spinner Verbs & Tips \(extracted from CLI binary v2.1.121\)](#)



📺 VIDEOS / PODCASTS

Video / Podcast	Source	YouTube
From Vibe Coding to Agentic Engineering (Andrej) 02 May 2026 AI Engineer	Karpathy	YouTube
Full Walkthrough: Workflow for AI Coding (Matt) 24 Apr 2026 Matt Pocock	Matt	YouTube
Everything We Got Wrong About Research-Plan-Implement (Dex) 24 Mar 2026 MLOps Community	Dex	YouTube
Building Claude Code with Boris Cherny (Boris) 04 Mar 2026 The Pragmatic Engineer	Boris	YouTube
Head of Claude Code: What happens after coding is solved (Boris) 19 Feb 2026 Lenny's Podcast	Boris	YouTube
Inside Claude Code With Its Creator Boris Cherny (Boris) 17 Feb 2026 Y Combinator	Boris	YouTube
Boris Cherny (Creator of Claude Code) On What Grew His Career (Boris) 15 Dec 2025 Ryan Peterman	Boris	YouTube
The Secrets of Claude Code From the Engineers Who Built It (Cat) 29 Oct 2025 Every	Boris	YouTube



🔔 SUBSCRIBE

Source	Name	Badge
	r/ClaudeAI , r/ClaudeCode , r/Anthropic	
	Claude , Claude Devs , Anthropic , Boris , Thariq , Cat , Lydia , Noah , Anthony , Alex , Kenneth	
	Jesse Kriss (Superpowers), Affaan Mustafa (ECC), Garry Tan (gstack), Dex Horthy (HumanLayer), Kieran Klaassen (Compound Eng), Tabish Gilani (OpenSpec), Brian McAdams (BMAD), Lex Christopherson (GSD), Matt Pocock (Skills), Dani Avila (CC Templates), Dan Shipper (Every), Andrej Karpathy (AutoResearch), Peter Steinberger (OpenClaw), Sigrid Jin (claw-code), Yeachen Heo (oh-my-claudecode)	
	Anthropic	
	Lenny's Podcast , Y Combinator , The Pragmatic Engineer , Ryan Peterman , Every , MLOps Community	



💀 STARTUPS / BUSINESSES

Claude	Replaced
Code Review	Greptile , CodeRabbit , Devin Review , OpenDiff , Cursor BugBot
Voice Dictation	Wispr Flow , SuperWhisper
Remote Control	OpenClaw
Claude in Chrome	Playwright MCP , Chrome DevTools MCP
Computer Use	OpenAI CUA
Cowork	ChatGPT Agent , Perplexity Computer , Manus
Tasks	Beads
Plan Mode	Agent OS

Claude	Replaced
Design	Figma , Framer , Sketch , v0
Agent SDK	LangChain , LangGraph , CrewAI , AutoGen , OpenAI Assistants API
Skills / Plugins	YC AI wrapper startups (reddit)



◆ Billion-Dollar Questions ◆

If you have answers, do let me know at shanraishan@gmail.com

Memory & Instructions (4)

- 1. What exactly should you put inside your CLAUDE.md – and what should you leave out?
- 2. If you already have a CLAUDE.md, is a separate constitution.md or rules.md actually needed?
- 3. How often should you update your CLAUDE.md, and how do you know when it's become stale?
- 4. Why does Claude still ignore CLAUDE.md instructions – even when they say MUST in all caps? ([reddit](#))

Agents, Skills & Workflows (6)

- 1. When should you use a command vs an agent vs a skill – and when is vanilla Claude Code just better?
- 2. How often should you update your agents, commands, and workflows as models improve?
- 3. Should you have a generalist subagent or a feature-specific/role-specific agent? Does giving your subagent a detailed persona improve quality, and what does a "perfect persona prompt" for research/vision look like?
- 4. Should you rely on Claude Code's built-in plan mode – or build your own planning command/agent that enforces your team's workflow?
- 5. If you have a personal skill (e.g., /implement with your coding style), how do you incorporate community skills (e.g., /simplify) without conflicts – and who wins when they disagree?
- 6. Are we there yet? Can we convert an existing codebase into specs, delete the code, and have AI regenerate the exact same code from those specs alone?

Specs & Documentation (3)

- 1. Should every feature in your repo have a spec as a markdown file?
- 2. How often do you need to update specs so they don't become obsolete when a new feature is implemented?
- 3. When implementing a new feature, how do you handle the ripple effect on specs for other features?

🔗 🤔 Does code matter?



🔗 REPORTS

AGENT SDK VS CLI	BROWSER AUTOMATION MCP	GLOBAL VS PROJECT SETTINGS	SKILLS IN MONOREPOS
AGENT MEMORY	ADVANCED TOOL USE	USAGE & RATE LIMITS	AGENTS VS COMMANDS VS SKILLS
LLM DEGRADATION	WHY HARNESS IS IMPORTANT	SPINNER VERBS & TIPS	

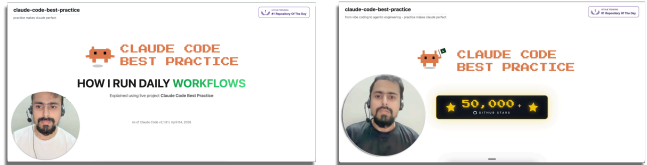


🔗 How to Use

Get the maximum out of this repo by following these steps:

1. **Read this repo as a course, not as a workflow or skill.** It's reference material first; you'll run things later.
2. **Don't use Claude as a chatbot.** Learn the primitives — agents, commands, skills, hooks — and assemble them into your own workflow.
3. **Run [/weather-orchestrator](#)** to see a complete command → agent → skill flow. Use it as a template for any dev workflow, from planning to shipping.
4. **Listen for the custom hook sounds while you work.** Their implementation lives in the dedicated [Claude Code Hooks repo](#); other patterns like [Agent Teams](#) ship inside this repo's `implementation/` directory.
5. **Learn the advanced topics and their implementations** from the 🔥 [Hot](#) sub-table — for example, the [Ralph Wiggum self-evolving loop](#) is a full working repo you can clone to see one of these patterns end-to-end.
6. **Point Claude at the [tips and tricks](#) section in your own project** and ask it to suggest edits — especially how to restructure your `CLAUDE.md`. Every tip is sourced from the Claude team or the community.
7. **Subscribe to the Reddit and YouTube channels in the [Subscribe section](#)** to keep up with the community.

Videos



Presentations



 github.com/trending?since=monthly

Trending

See what the GitHub community is most excited about this month.

Repositories

Developers

Spoken Language: Any ▾ Language: Any ▾ Date range: This month ▾

 [shanraishan / claude-code-best-practice](#)

☆ Star ▾

practice made claude perfect

HTML

☆ 14,150

🔗 1,330

Built by    

☆ 11,250 stars this month

🌟 Trending on Github in March 2026 🌟

Star History

 [Star History Chart](#)

★ 64k stars and counting

🔗 Other Repos


[Claude Code Hooks](#)


[Codex CLI Best Practice](#)


[Codex CLI Hooks](#)


[Gemini CLI Best Practice](#)


[Gemini CLI Hooks](#)

🔗 Developed by

🌟 This project is 100% autopilot and developed by Claude Code

#	Workflow	Description
1	/workflows:development-workflows	Update the DEVELOPMENT WORKFLOWS table and cross-workflow analysis report by researching all 10 workflow repos in parallel
2	/workflows:skill-collections	Update the SKILL COLLECTIONS table by researching all 5 skill-collection repos in parallel
3	/workflows:agent-collections	Update the AGENT COLLECTIONS table by researching all agent-collection repos in parallel
4	/workflows:best-practice:workflow-concepts	Update the README CONCEPTS section with the latest Claude Code features and concepts
5	/workflows:best-practice:workflow-claude-settings	Track Claude Code settings report changes and find what needs updating
6	/workflows:best-practice:workflow-claude-subagents	Track Claude Code subagents report changes and find what needs updating
7	/workflows:best-practice:workflow-claude-commands	Track Claude Code commands report changes and find what needs updating
	/workflows:best-practice:workflow-	

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors

No contributors

Languages

HTML 95.5% Python 4.5%

Suggested workflows

Based on your tech stack



Jekyll using Docker image
Package a Jekyll site using the jekyll/builder Docker image.
By GitHub Actions

Configure



SLSA Generic generator

Configure

Generate SLSA3 provenance for your existing release workflows
By Open Source Security Foundation (OpenSSF)



Pylint

Lint a Python application with pylint.
By GitHub Actions

Configure

[More workflows](#)

